भारत अंतरिक्ष प्रयोगशाला
बीए/14बी, जनकपुरी, नई दिल्ली, भारत
दूरभाषः 011–44749707, 9211293116
ई–मेलः office@isl.ac.in, info@isl.ac.in
वेबसाइटः www.isl.ac.in

INDIA SPACE LAB
BA/14B, Janakpuri, New Delhi, India
Telephone: 011-44749707, 9211293116
E-mail: office@isl.ac.in, info@isl.ac.in
Website: www.isl.ac.in

cansat and cubesat satellite
Project work

Comprehensive Assignment Brief: Development of a Single-Page CanSat Ground Control Software (GCS)

Course Domain: Aerospace Engineering / Embedded Systems / Avionics / Ground Systems
Assignment Type: Design and Development Assignment
Project Category: Real-Time Telemetry Monitoring and Mission Operations Software

Assignment Overview

In this assignment, students are required to design and develop a professional single-page Ground Control Software (GCS) for a CanSat mission. The software should monitor telemetry data in real time, visualize mission parameters, display GPS tracking, handle mission controls, and simulate aerospace mission operations. Students are expected to design the interface in a modular, readable, and mission-oriented manner similar to professional aerospace systems.

1. Interface Layout

Objective: Design a professional aerospace-style operator dashboard.

●Background Knowledge:

- A Ground Control Software dashboard acts as the main monitoring interface for mission operators.
- The interface should help operators quickly identify mission health, telemetry, warnings, and mission state.
- Real aerospace systems prioritize readability, spacing,

- **Instructions:**
  - Design the dashboard as a single-page interface.
  - Divide sections into telemetry, graphs, controls, tracking map, orientation visualization, and video stream.
  - Use consistent fonts, colors, spacing, and alignment.
  - Ensure the interface supports smooth real-time updates.

Suggested Tools: HTML5, CSS3, CSS Grid, Flexbox, JavaScript

- Documentation Links:
  - https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout
  - https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout/Flexbox

2. Top Control Bar

Objective: Implement mission operation controls.

- Background Knowledge:

The top control bar is used to manage telemetry reception and mission operations.

Operators use this section to start/stop telemetry and export mission data.

- **Instructions:**
  - Add Start and Stop buttons for telemetry streaming.
  - Add Export CSV and Export Graph buttons.
  - Add Sync PC Time and Reset Packet buttons.

Suggested Tools: JavaScript DOM Manipulation

●Documentation Links:

https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

3. Mission Control Panel

Objective: Implement mission-critical control operations.

●**Background Knowledge:**

Mission controls are responsible for sending commands to the CanSat.

Critical commands must provide clear feedback and warning indications.

●**Instructions:**

- Add Manual Separation control.
- Add Emergency Parachute Deployment control.
- Add Redundant Activation command.
- Display command execution status dynamically.

Suggested Tools: JavaScript, Web Serial API

●Documentation Links:

https://developer.mozilla.org/en-US/docs/Web/API/Web_Serial_API

4. Telemetry Display

Objective: Display telemetry packets accurately in real time.

- Background Knowledge:
  - Telemetry packets contain mission sensor data transmitted from the CanSat.
  - The GCS should continuously parse and display telemetry fields.
- **Instructions:**
  - Receive telemetry packets continuously.
  - Parse packet fields properly.
  - Display container telemetry and payload telemetry separately.
  - Update telemetry values continuously.

Suggested Tools: JavaScript String Parsing, WebSocket, Web Serial API

- Documentation Links:

https://developer.mozilla.org/en-US/docs/Web/API/WebSocket

5. Error Code System

Objective: Implement live mission fault monitoring.

- Background Knowledge:
  - The 4-digit error code system is used to quickly identify mission faults.
  - Each digit represents a different mission condition.
  - 0 means NO ERROR and 1 means ERROR/FAULT condition.
- Instructions:
  - Monitor all telemetry continuously.
  - Update error digits dynamically.
  - Use color-coded indicators for warnings and faults.

Suggested Tools: JavaScript Conditional Logic, CSS Animation

- Documentation Links:

https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Building_blocks/conditionals.

Detailed Error Code Explanation

The Ground Control Software must implement a 4-digit error code system. Each digit represents a specific mission condition.

Digit	Condition	0 Meaning	1 Meaning
1	Descent Rate	Descent rate is within 8–10 m/s	Descent rate is outside safe range
2	GPS Availability	GPS data available	GPS data unavailable
3	Payload Separation	Payload separated successfully	Payload separation failure
4	Emergency Parachute	Parachute inactive	Emergency parachute activated

Example:
0000 → All systems normal
1000 → Descent rate fault detected
0100 → GPS data unavailable
0010 → Payload separation failure
0001 → Emergency parachute activated
1111 → All fault conditions active

6. Real-Time Graphs

Students should create continuously updating graphs for altitude, pressure, temperature, descent rate, and battery voltage using Chart.js or similar graphing libraries. The graphs should update smoothly in real time and support telemetry monitoring usability.

Suggested Tools: Chart.js, HTML Canvas

Documentation: https://www.chartjs.org/docs/latest/

7. Tracking Map

Students should display live payload GPS coordinates on a real-time tracking map using Leaflet.js and OpenStreetMap. The map should update continuously and display mission trajectory/path history.

Suggested Tools: Leaflet.js, OpenStreetMap

Documentation: https://leafletjs.com/reference.html

8. Orientation Visualization

Students should implement a dynamic orientation visualization using Roll, Pitch, and Yaw telemetry values. The visualization may include an artificial horizon or a 3D orientation model using Three.js.

Suggested Tools: Three.js, WebGL

Documentation: https://threejs.org/docs/

9. Live Video Streaming

Students should integrate live video streaming support using browser camera APIs. The interface should support camera selection, stream start/stop controls, and stream status indication.

Suggested Tools: MediaDevices API, HTML Video

Documentation: https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia

10. Data Management Features

Students should implement telemetry logging, CSV export, graph export, packet reset, and telemetry storage functionalities. Exported files should remain organized and readable.

Suggested Tools: Blob API, File API

Documentation: https://developer.mozilla.org/en-US/docs/Web/API/Blob

11. Testing Strategy

Students must use the microcontroller received in the **WeGyanik** Kit as a real-time telemetry streaming device. The microcontroller should generate dummy telemetry packets and continuously transmit telemetry data to the PC. Students should test telemetry reception, graph updates, error handling, map tracking, and orientation visualization under multiple simulated mission conditions.

Suggested Tools: Arduino IDE, Web Serial API, PySerial

Documentation: https://docs.arduino.cc/

Deliverables

- Ground Control Software Source Code
- Executable/Application Files
- Project Documentation/Report
- UI Screenshots
- Demonstration Video
- Sample Telemetry Logs
- Exported CSV Samples
- Graph Export Samples

भारत अंतरिक्ष प्रयोगशाला
बीए/14बी, जनकपुरी, नई दिल्ली, भारत
दूरभाषः 011–44749707, 9211293116
ई–मेलः office@isl.ac.in, info@isl.ac.in
वेबसाइटः www.isl.ac.in

INDIA SPACE LAB
BA/14B, Janakpuri, New Delhi, India
Telephone: 011-44749707, 9211293116
E-mail: office@isl.ac.in, info@isl.ac.in
Website: www.isl.ac.in

Evaluation Criteria

Criteria	Weightage
UI/UX Design	15%
Telemetry Handling	20%
Real-Time Visualization	20%
Mission Control Features	15%
Graphing and Tracking	10%
Orientation and Video Systems	10%
Code Quality and Scalability	10%